1. Loading and Sorting Data

The dataset is first loaded and sorted by device and time. This is critical because time-series operations depend on chronological order.

```
df = pd.read_csv(INPUT_FILE)
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values(['client_mac', 'timestamp'])
```

2. Time Delta Computation

We compute the time difference between consecutive samples for each device. This establishes temporal spacing and enables rate calculations.

```
df['time_delta'] = df.groupby('client_mac')['timestamp'].diff().dt.total_seconds()
```

3. Counter to Rate Conversion

WiFi stats are cumulative counters. We convert them into rates using differencing. This creates meaningful time-series signals.

```
df['tx_packet_rate'] = df.groupby('client_mac')['tx_data_packets'].diff() / df['time_delta']
df['rx_packet_rate'] = df.groupby('client_mac')['rx_data_packets'].diff() / df['time_delta']
```

4. Activity Detection

The is_active flag identifies whether meaningful traffic occurred during the interval.

```
tx_delta = df.groupby('client_mac')['tx_data_packets'].diff().fillna(0)
rx_delta = df.groupby('client_mac')['rx_data_packets'].diff().fillna(0)

df['is_active'] = ((tx_delta > 0) | (rx_delta > 0)).astype(int)
```

5. Reliability Metrics

These features quantify network quality and transmission behavior.

```
df['tx_success_rate'] = tx_success_delta / tx_total_delta
df['retry_rate'] = retry_delta / tx_total_delta
df['tx_failed_rate'] = failed_delta / tx_total_delta
df['rx_error_rate'] = rx_errors_delta / rx_packets_delta
```

6. Throughput Features

Throughput represents how fast data is transmitted and received.

```
df['tx_throughput_mbps'] = df['average_tx_rate_kbps'] / 1000
df['rx_throughput_mbps'] = df['average_rx_rate_kbps'] / 1000
```

7. MCS Feature Engineering

MCS values capture link quality. We derive ratios, weighted score, and entropy to represent stability and performance.

```
low_cols = [c for c in mcs_cols if 0 <= get_mcs_index(c) <= 3]
mid_cols = [c for c in mcs_cols if 4 <= get_mcs_index(c) <= 7]
high_cols = [c for c in mcs_cols if get_mcs_index(c) >= 8]

df['high_mcs_ratio'] = df[high_cols].sum(axis=1) / total_mcs
```

8. Data Cleaning

Invalid values are handled to ensure compatibility with machine learning models.

```
df = df.replace([np.inf, -np.inf], np.nan)
df = df.fillna(0)
```

9. Final Dataset

The final dataset contains selected features where each row represents a device state at a specific time.

```
df_final = df[final_columns]
df_final.to_csv(OUTPUT_FILE, index=False)
```